



PERTEMUAN 12

## Strings, Characters & Regular Expressions

Lanjutan pembahasan tipe data karakter, ekspresi reguler, dan pengantar rekursi dalam pemrograman Java. Materi ini akan membekali mahasiswa dengan kemampuan memproses teks secara efisien dan memahami pola-pola dalam data.



# Class Character

Class **Character** dalam Java menyediakan sekumpulan **metode statis** yang sangat berguna untuk memeriksa dan memanipulasi karakter. Bayangkan karakter seperti sebuah benda – class ini membantu kita menjawab pertanyaan: "Apakah ini huruf? Angka? Huruf besar atau kecil?"

## Pengujian Tipe

Metode seperti `isLetter()`, `isDigit()`, `isWhitespace()` menguji apakah karakter termasuk kategori tertentu.

## Konversi Huruf

Metode `toUpperCase()` dan `toLowerCase()` mengubah huruf kecil menjadi huruf besar dan sebaliknya secara instan.

## Metode Statis

Semua metode dipanggil langsung dari nama class, tanpa perlu membuat objek: `Character.isLetter('A')`



# Konversi Digit & Sistem Bilangan

Class Character juga menyediakan metode

# Character Class — Instance Methods

Selain metode statis, class **Character** juga memiliki instance methods yang bisa dipanggil pada objek Character. Berikut adalah ringkasan metode-metode penting yang perlu dikuasai:

| Metode                        | Fungsi                                | Contoh                                |
|-------------------------------|---------------------------------------|---------------------------------------|
| <code>isLetter(ch)</code>     | Mengecek apakah karakter adalah huruf | <code>isLetter('A') → true</code>     |
| <code>isDigit(ch)</code>      | Mengecek apakah karakter adalah digit | <code>isDigit('5') → true</code>      |
| <code>isUpperCase(ch)</code>  | Mengecek apakah huruf kapital         | <code>isUpperCase('Z') → true</code>  |
| <code>isLowerCase(ch)</code>  | Mengecek apakah huruf kecil           | <code>isLowerCase('a') → true</code>  |
| <code>isWhitespace(ch)</code> | Mengecek apakah spasi/tab/newline     | <code>isWhitespace(' ') → true</code> |
| <code>toUpperCase(ch)</code>  | Mengubah ke huruf besar               | <code>toUpperCase('a') → 'A'</code>   |
| <code>toLowerCase(ch)</code>  | Mengubah ke huruf kecil               | <code>toLowerCase('A') → 'a'</code>   |



# Tokenizing Strings — Memecah String Menjadi Bagian

**Tokenisasi** adalah proses memecah sebuah teks panjang menjadi bagian-bagian kecil yang bermakna, disebut **token**. Ini mirip cara kompiler membaca kode program dan memisahkannya menjadi kata kunci, variabel, dan operator.

## Apa itu Token?

Token adalah "potongan" dari sebuah String yang dipisahkan oleh karakter pembatas (delimiter). Contoh: kalimat "Halo Dunia Java" memiliki 3 token jika spasi adalah delimiternya.

## Metode split()

Class String menyediakan metode `split()` yang memecah String menjadi array token. Sintaks:

```
String[] tokens = kalimat.split(" ");  
// Memecah berdasarkan spasi
```

Delimiter bisa berupa karakter apa saja: koma, titik koma, tab, bahkan ekspresi reguler.

- ❗ Bayangkan tokenisasi seperti memotong kalimat dengan gunting di setiap tanda spasi atau koma — hasilnya adalah kata-kata individual yang siap diproses satu per satu.

# Regular Expressions — Ekspresi Reguler

**Ekspresi Reguler (Regex)** adalah pola teks yang digunakan untuk mencari, mencocokkan, dan memvalidasi String. Seperti sebuah "template" yang menggambarkan seperti apa teks yang valid itu.

## 1 Validasi Format Data

Memastikan input pengguna sesuai format — misalnya kode pos harus 5 digit, email harus mengandung @, nomor telepon hanya berisi angka dan tanda hubung.

## 2 Pencarian Pola dalam Teks

Mencari semua kata yang diawali huruf kapital, atau menemukan semua URL dalam sebuah dokumen panjang secara otomatis.

## 3 Validasi Sintaks Program

Kompiler menggunakan ekspresi reguler yang kompleks untuk memeriksa apakah kode program ditulis dengan sintaks yang benar. Jika tidak cocok, kompiler melaporkan kesalahan.



# Predefined Character Classes — Kelas Karakter Bawaan

Java menyediakan **kelas karakter bawaan** dalam ekspresi reguler yang mempermudah penulisan pola. Alih-alih menulis pola yang panjang, kita bisa menggunakan singkatan khusus ini:

**`\d`**

Digit (0–9). Ekuivalen dengan `[0-9]`. Contoh: `\d{5}` cocok dengan 5 digit berturut-turut (seperti kode pos).

**`\w`**

Word character — huruf, digit, atau underscore. Ekuivalen dengan `[a-zA-Z0-9_]`. Berguna untuk validasi username.

**`\s`**

Whitespace — spasi, tab, newline. Digunakan untuk mendeteksi pemisah antar kata dalam teks.

**`.`**

Karakter apa saja (kecuali newline). Tanda titik dalam regex adalah wildcard yang cocok dengan semua karakter.

**`[abc]`**

Karakter set — hanya cocok dengan salah satu dari a, b, atau c. Bisa dikombinasikan: `[a-z]` untuk semua huruf kecil.

# Validasi Input Pengguna dengan Regex

Salah satu penggunaan paling umum ekspresi reguler adalah **memvalidasi input** yang dimasukkan pengguna. Sebelum data disimpan atau diproses, kita perlu memastikan formatnya benar.



## Validasi Email

Pola regex untuk email:

```
^[\\w. - ]+@[\\w. -
```

```
] + \\. \\w{2, }$.
```

 Memastikan ada karakter sebelum @, nama domain, dan ekstensi minimal 2 huruf.

## Validasi Nomor Telepon

Pola: `^\d{10,13}$` –

memastikan nomor telepon terdiri dari 10 hingga 13 digit angka saja, tanpa huruf atau simbol lain.



## Validasi Password

Pola: `^(?=.*[A-`

```
Z])(?=.*\\d)\\. {8, }$
```

 –

memastikan password mengandung minimal 1 huruf besar, 1 angka, dan panjang minimal 8 karakter.

# Input & Validasi dengan Class **ValidateInput**

Dalam praktik pemrograman Java, kita dapat membuat class khusus bernama **ValidateInput** yang mengelompokkan semua logika validasi berbasis regex. Pendekatan ini membuat kode lebih terorganisir dan mudah digunakan ulang.

## Struktur Class **ValidateInput**

Class ini biasanya berisi metode-metode statis yang masing-masing memvalidasi satu jenis data:

- `validateFirstName(String s)` – hanya huruf dan spasi
- `validateLastName(String s)` – huruf, spasi, apostrof, tanda hubung
- `validatePhone(String s)` – format nomor telepon
- `validateEmail(String s)` – format email yang valid
- `validateZip(String s)` – 5 digit kode pos

## Cara Kerja Validasi

Setiap metode menggunakan `String.matches(regex)` untuk memeriksa apakah input sesuai pola. Metode ini mengembalikan `true` jika cocok, `false` jika tidak.

```
public static boolean validateZip(  
    String zip) {  
    return zip.matches("\\d{5}");  
}
```

Penggunaan class ini membuat validasi di program utama menjadi sangat ringkas dan mudah dibaca.

# Replacing Substrings & Splitting Strings

Selain pencarian pola, ekspresi reguler juga digunakan untuk **mengganti** dan **memecah** String. Class String menyediakan tiga metode utama untuk keperluan ini.

## `replaceAll(regex, replacement)`

Mengganti **semua** kemunculan pola dengan teks pengganti. Contoh: menghapus semua angka dari sebuah String, atau mengganti semua spasi ganda dengan satu spasi.

```
"Halo  
Dunia".replaceAll("\\s+", "  
")  
// Hasil: "Halo Dunia"
```

## `replaceFirst(regex, replacement)`

Mengganti hanya **kemunculan pertama** dari pola. Berguna ketika kita hanya ingin mengubah satu bagian tanpa memengaruhi yang lain.

```
"aababab".replaceFirst("a",  
"X")  
// Hasil: "Xababab"
```

## `split(regex)`

Memecah String menjadi array berdasarkan pola pembatas. Lebih fleksibel dari split biasa karena bisa menggunakan pola regex sebagai delimiter.

```
"a,b,,c".split(",")  
// Hasil: ["a","b","","c"]
```



# Classes **Pattern** dan **Matcher**

Untuk kebutuhan regex yang lebih kompleks, Java menyediakan class **Pattern** dan **Matcher** dalam paket `java.util.regex`. Keduanya bekerja bersama untuk pencarian pola yang lebih canggih dan efisien.

## Class **Pattern**

**Pattern** merepresentasikan ekspresi reguler yang sudah dikompilasi. Membuat **Pattern** sekali lalu menggunakannya berkali-kali jauh lebih efisien daripada memanggil `matches()` berulang kali.

```
Pattern p =  
Pattern.compile("\\d+");
```

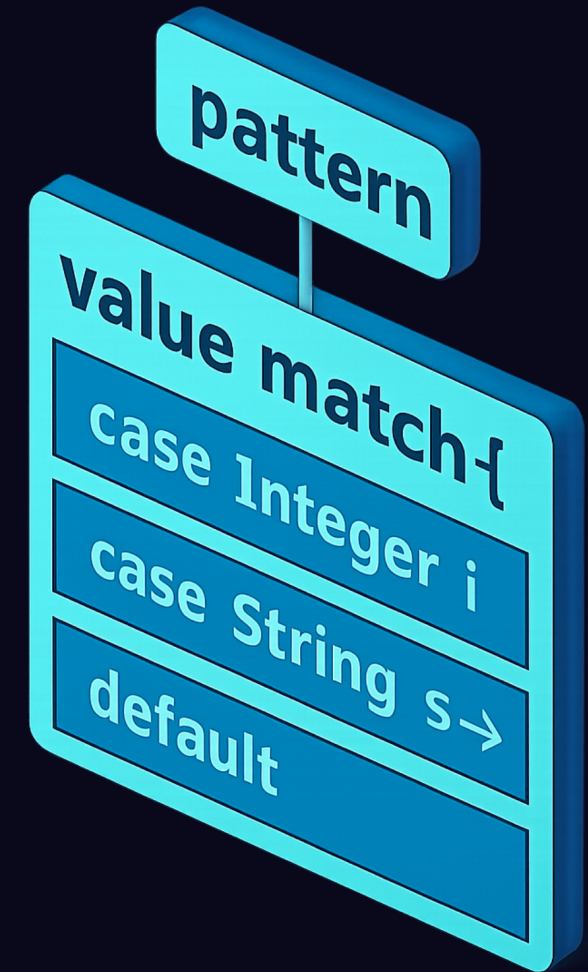
**Pattern** bekerja dengan antarmuka **CharSequence**, sehingga kompatibel dengan `String` maupun `StringBuilder`.

## Class **Matcher**

**Matcher** adalah mesin pencari yang menggunakan **Pattern** untuk mencari pola dalam teks. **Matcher** menyediakan kemampuan pencarian yang tidak ada di method `String` biasa.

```
Matcher m =  
p.matcher("abc123def456");  
while (m.find()) {  
  
    System.out.println(m.group());  
}  
  
// Output: 123, 456
```

Metode `find()` mencari kemunculan berikutnya, `group()` mengembalikan teks yang cocok.



# Mengenal Rekursi

**Rekursi** adalah teknik pemrograman di mana sebuah metode memanggil dirinya sendiri untuk memecahkan masalah. Ini terdengar membingungkan, tapi analoginya sederhana: bayangkan Anda berdiri di antara dua cermin yang saling berhadapan – refleksi yang tercipta adalah contoh konsep rekursi.

## Pemanggilan Langsung

Metode A memanggil dirinya sendiri secara eksplisit: `methodA() { ... methodA(); }`

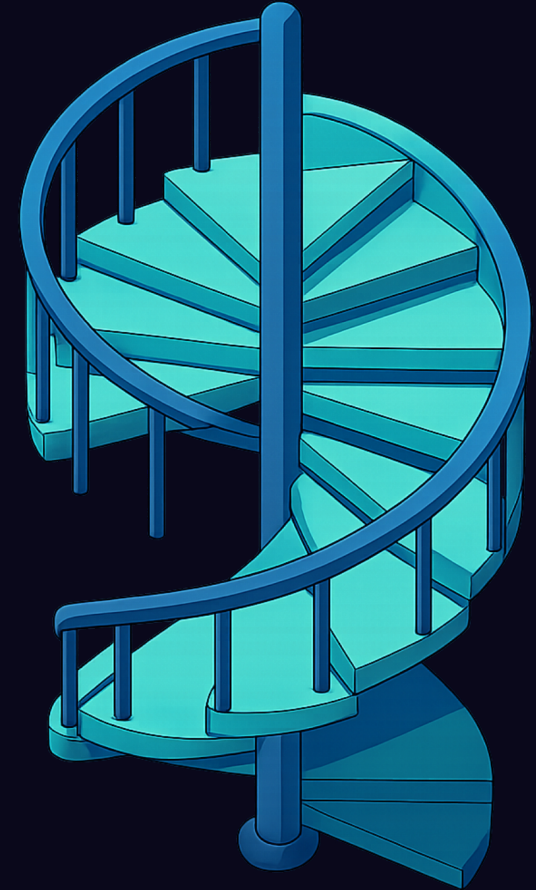
## Pemanggilan Tidak Langsung

Metode A memanggil Metode B, dan Metode B memanggil kembali Metode A membentuk siklus rekursi.

## Pentingnya dalam CS

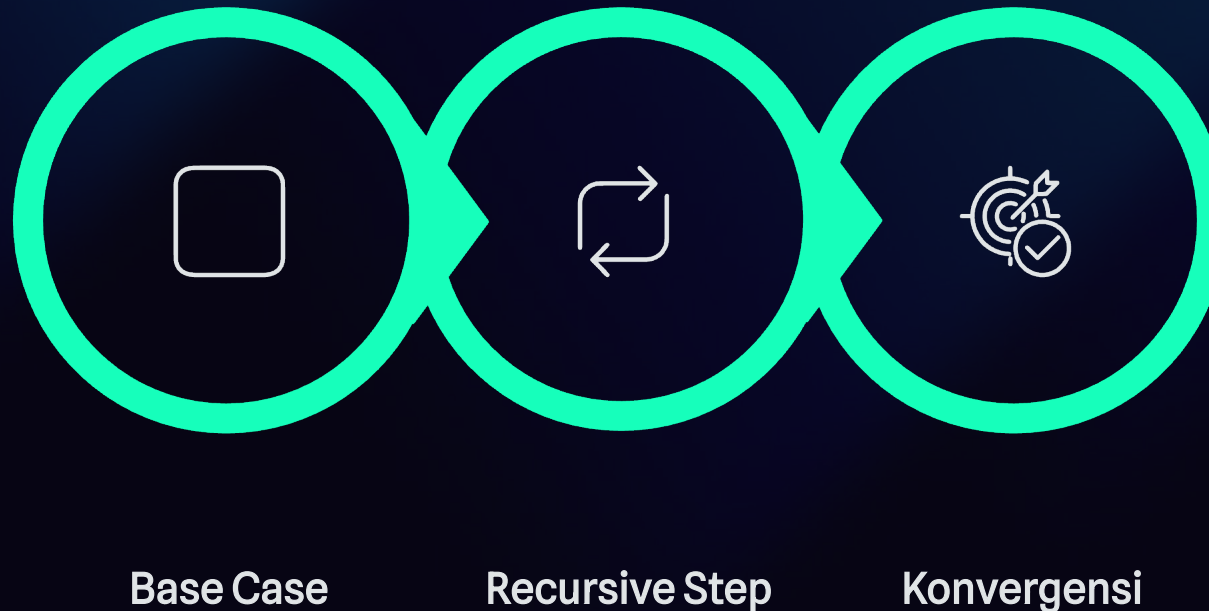
Rekursi adalah fondasi penting dalam algoritma lanjut seperti sorting, tree traversal, dan pemecahan puzzle.

- ❗ Rekursi tidak berarti program berjalan selamanya – ada titik berhenti yang disebut **base case** yang mengakhiri pemanggilan berulang.



# Konsep Rekursi — Cara Kerjanya

Setiap metode rekursif bekerja dengan dua bagian utama: **Base Case** (kondisi berhenti) dan **Recursive Step** (langkah yang menyederhanakan masalah). Tanpa keduanya, rekursi tidak akan berfungsi dengan benar.



Kunci rekursi: setiap pemanggilan rekursif harus membuat masalah menjadi **satu langkah lebih kecil** dari masalah sebelumnya, sehingga pada akhirnya mencapai base case dan berhenti. Jika rekursi tidak semakin mengecil, akan terjadi **stack overflow**.

# Rekursi: Contoh Faktorial

Faktorial adalah contoh klasik rekursi. Faktorial dari  $n$  (ditulis  $n!$ ) adalah hasil perkalian semua bilangan bulat positif dari 1 hingga  $n$ . Misalnya,  $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$ .

## Definisi Matematis

$$n! = \begin{cases} 1 & \text{jika } n = 0 \text{ atau } n = 1 \\ n \times (n-1)! & \text{jika } n > 1 \end{cases}$$

Base case:  $0! = 1$  dan  $1! = 1$

Recursive step:  $n! = n \times (n-1)!$

## Implementasi Java

```
public static long factorial(long n) {  
    if (n == 0 || n == 1) // base case  
        return 1;  
    else // recursive step  
        return n * factorial(n - 1);  
}  
  
// Contoh pemanggilan:  
// factorial(5)  
// = 5 * factorial(4)  
// = 5 * 4 * factorial(3)  
// = 5 * 4 * 3 * 2 * 1  
// = 120
```

# BigInteger untuk Faktorial Besar

Tipe data `long` memiliki batas maksimum sekitar  $9,2 \times 10^{18}$ . Untuk faktorial bilangan besar (misalnya  $25! = 15.511.210.043.330.985.984.000.000$ ), kita perlu menggunakan class **BigInteger** dari paket `java.math`.

## Mengapa BigInteger?

BigInteger dapat menyimpan bilangan bulat dengan ukuran tak terbatas (hanya dibatasi memori). Class ini sangat berguna untuk komputasi kriptografi dan matematika presisi tinggi.

- Tidak ada batas ukuran bilangan
- Operasi aritmatika lengkap tersedia
- Immutable — aman untuk konkurensi

## Implementasi dengan BigInteger

```
import java.math.BigInteger;

public static BigInteger factorial(
    BigInteger n) {
    if (n.compareTo(BigInteger.ONE) <= 0)
        return BigInteger.ONE;
    else
        return n.multiply(
            factorial(n.subtract(BigInteger.ONE))
        );
}

// factorial(BigInteger.valueOf(25))
// menghasilkan angka yang sangat besar!
```

 Gunakan `BigInteger.valueOf(n)` untuk mengkonversi `long` ke `BigInteger`, dan `BigInteger.ONE`, `BigInteger.ZERO` untuk konstanta umum.

# Rekursi: Deret Fibonacci

Deret Fibonacci adalah barisan angka di mana setiap angka adalah penjumlahan dua angka sebelumnya: **0, 1, 1, 2, 3, 5, 8, 13, 21, 34...** Deret ini ditemukan di alam – dari spiral kerang hingga susunan biji bunga matahari.

## Definisi Rekursif

$$F(n) = \begin{cases} 0 & \text{jika } n = 0 \\ 1 & \text{jika } n = 1 \\ F(n-1) + F(n-2) & \text{jika } n > 1 \end{cases}$$

Ada **dua** base case:  $F(0)=0$  dan  $F(1)=1$ , karena setiap langkah rekursif membutuhkan dua pemanggilan sebelumnya.

## Implementasi Java

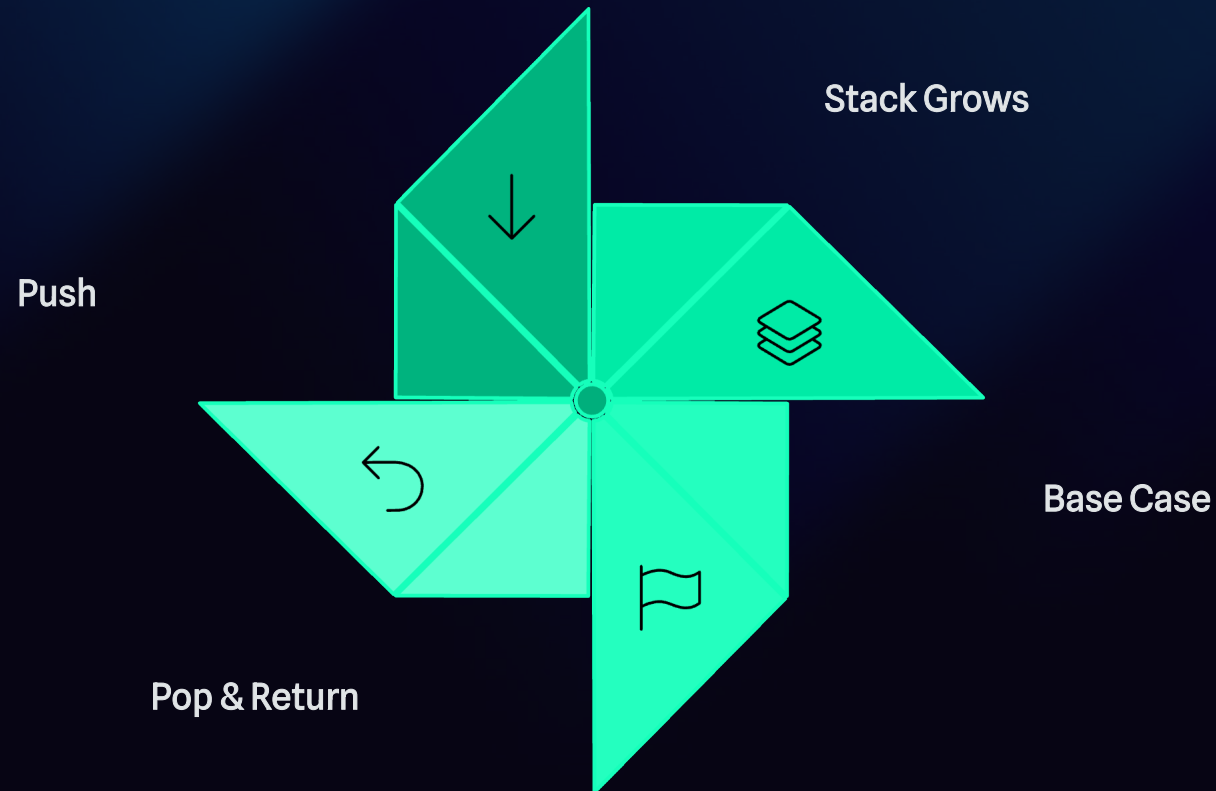
```
public static long fibonacci(long n) {  
    if (n == 0)        // base case 1  
        return 0;  
    else if (n == 1) // base case 2  
        return 1;  
    else                // recursive step  
        return fibonacci(n-1) + fibonacci(n-2);  
}  
  
// fibonacci(7) = 13  
// fibonacci(10) = 55
```

⚠ Implementasi rekursif Fibonacci ini kurang efisien untuk  $n$  besar karena terjadi banyak pemanggilan yang berulang-ulang (exponential time).



# Rekursi dan Method-Call Stack

Saat metode rekursif berjalan, setiap pemanggilan menyimpan konteksnya di **call stack** – sebuah struktur data LIFO (Last In, First Out) di memori. Memahami stack penting untuk mengerti bagaimana rekursi bekerja dan mengapa bisa menyebabkan error.



Setiap frame di stack menyimpan: nilai parameter lokal, variabel lokal, dan alamat kembali. Jika rekursi terlalu dalam (terlalu banyak pemanggilan sebelum base case), akan terjadi **StackOverflowError** karena memori stack habis.

# Rekursi vs. Iterasi — Perbandingan Mendalam

Rekursi dan iterasi (loop) adalah dua cara berbeda untuk menyelesaikan masalah berulang. Masing-masing memiliki kelebihan dan kekurangan yang perlu dipahami sebelum memilih pendekatan yang tepat.

## Iterasi

- Menggunakan `for`, `while`, atau `do-while`
- Lebih efisien dalam penggunaan memori
- Tidak ada risiko `StackOverflow`
- Kode kadang lebih panjang tapi lebih eksplisit
- Cocok untuk masalah sederhana yang berulang

## Rekursi

- Metode memanggil dirinya sendiri
- Membutuhkan memori stack lebih besar
- Risiko `StackOverflowError` jika terlalu dalam
- Kode lebih elegan dan mudah dibaca untuk masalah tertentu
- Ideal untuk tree, graph, dan divide-and-conquer

 Aturan praktis: gunakan **rekursi** ketika struktur masalah secara alami rekursif (seperti tree traversal). Gunakan **iterasi** ketika performa dan efisiensi memori adalah prioritas utama.

# Rekursi vs. Iterasi — Kapan Memilih Yang Mana?



## Penghentian

Iterasi berhenti saat kondisi loop `false`. Rekursi berhenti saat base case tercapai. Keduanya bisa berjalan tak terbatas jika tidak dirancang dengan benar.



## Penggunaan Memori

Iterasi menggunakan memori konstan. Rekursi menggunakan memori proporsional dengan kedalaman panggilan — setiap frame stack membutuhkan alokasi memori baru.



## Performa

Iterasi umumnya lebih cepat karena tidak ada overhead pemanggilan metode. Rekursi memiliki biaya tambahan untuk setiap pemanggilan (push/pop stack frame).



## Keterbacaan Kode

Untuk masalah yang secara alami rekursif (faktorial, Fibonacci, tree), kode rekursif jauh lebih mudah dibaca dan dipahami dibanding versi iteratifnya.

# Studi Kasus: Towers of Hanoi

Towers of Hanoi adalah puzzle matematika klasik yang sangat baik diilustrasikan dengan rekursi. Tujuannya: pindahkan semua cakram dari tiang A ke tiang C, menggunakan tiang B sebagai pembantu, dengan aturan bahwa cakram yang lebih besar tidak boleh diletakkan di atas cakram yang lebih kecil.

## Aturan Permainan

1. Hanya satu cakram yang boleh dipindah dalam satu langkah
2. Cakram yang lebih besar tidak boleh berada di atas cakram yang lebih kecil
3. Semua cakram harus dipindah ke tiang tujuan

Untuk  $n$  cakram, jumlah langkah minimum adalah  $2^n - 1$ . Untuk 3 cakram = 7 langkah, untuk 64 cakram = miliaran tahun!

## Solusi Rekursif

```
public static void hanoi(
    int n, char from, char to, char aux) {
    if (n == 1) { // base case
        System.out.printf(
            "Pindah cakram 1 dari %c ke %c\n",
            from, to);
        return;
    }
    // Pindah n-1 cakram ke tiang pembantu
    hanoi(n-1, from, aux, to);
    System.out.printf(
        "Pindah cakram %d dari %c ke %c\n",
        n, from, to);
    // Pindah n-1 cakram dari pembantu ke tujuan
    hanoi(n-1, aux, to, from);
}
```



# Fraktal — Geometri **Rekursif**

**Fraktal** adalah bentuk geometris yang memiliki sifat **self-similar** — artinya, setiap bagian kecilnya menyerupai bentuk keseluruhannya. Fraktal dihasilkan dengan menerapkan pola yang sama secara rekursif, membuat setiap iterasi menjadi lebih detail.

## Asal Usul

Matematikawan **Benoit Mandelbrot** memperkenalkan istilah "fraktal" pada tahun 1970-an. Meskipun konsepnya sudah ada sebelum abad ke-20, Mandelbrot-lah yang memformalkan matematika di baliknya.

## Di Alam

Fraktal ditemukan di mana-mana di alam: kepingan salju, cabang pohon, garis pantai, pembuluh darah, bahkan pola petir — semuanya memiliki sifat self-similar fraktal.

## Aplikasi

Geometri fraktal Mandelbrot digunakan dalam pemodelan alam untuk grafis komputer, kompresi gambar, antena fraktal, dan pemodelan cuaca.

# Fraktal dalam Pemrograman Java

Dalam Java, fraktal digambar menggunakan rekursi bersama library grafis seperti **JavaFX** atau **Swing**. Setiap langkah rekursi menambahkan detail yang lebih kecil pada gambar, menciptakan kompleksitas visual dari aturan sederhana.

## Prinsip Menggambar Fraktal

1. Mulai dengan bentuk dasar (garis atau segitiga)
2. Terapkan transformasi pada setiap bagian bentuk
3. Panggil fungsi secara rekursif untuk menggambar subbagian
4. Hentikan rekursi saat level kedalaman tertentu tercapai

Parameter **level** atau **depth** mengontrol seberapa detail fraktal yang digambar. Semakin besar nilainya, semakin detail tapi semakin lama pula waktu komputasinya.

## Contoh Struktur Kode

```
public void drawFractal(
    GraphicsContext gc,
    double x1, double y1,
    double x2, double y2,
    int level) {
    if (level == 0) { // base case
        gc.strokeLine(x1, y1, x2, y2);
        return;
    }
    // Hitung titik tengah
    double midX = (x1 + x2) / 2;
    double midY = (y1 + y2) / 2;
    // Rekursif untuk subbagian
    drawFractal(gc, x1,y1, midX,midY, level-1);
    drawFractal(gc, midX,midY, x2,y2, level-1);
}
```

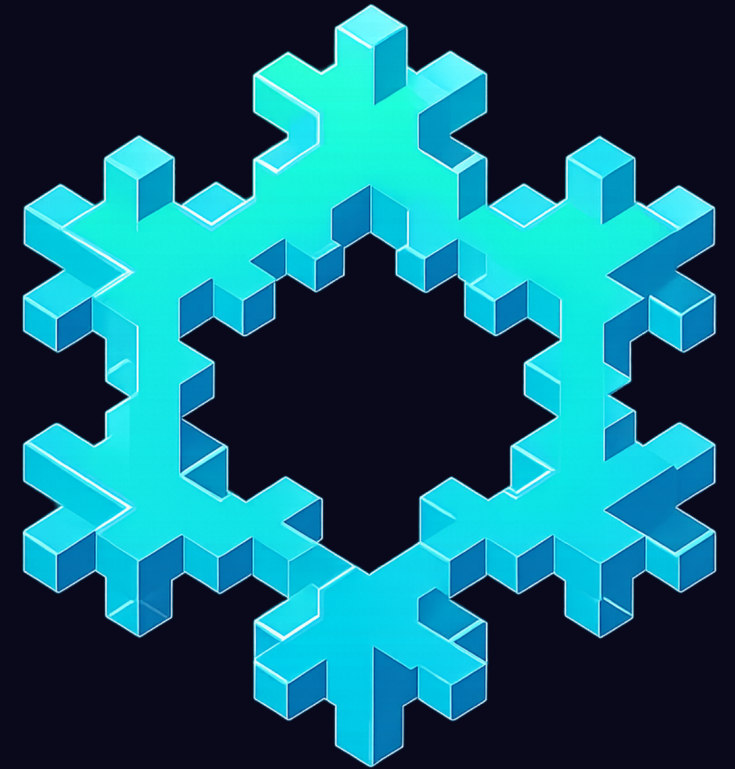


# Koch Curve Fractal — Kurva Koch

**Kurva Koch** adalah salah satu fraktal paling terkenal yang mendemonstrasikan bagaimana pola sederhana yang diulang dapat menciptakan bentuk yang sangat kompleks. Kurva ini ditemukan oleh matematikawan Swedia Helge von Koch pada tahun 1904.



Sifat menarik Kurva Koch: panjangnya **tidak terbatas** (semakin bertambah setiap iterasi), namun luasnya **terbatas**. Ini adalah paradoks matematika yang elegan — bentuk dengan keliling tak hingga namun luas yang terhingga. Dalam implementasi Java, kedalaman rekursi menentukan seberapa detail kurva yang dihasilkan.



# Terima Kasih! 🎓

Selamat! Anda telah menyelesaikan materi **Pertemuan 12** tentang Strings, Characters, Regular Expressions, dan pengantar Rekursi.

01

## Class Character

Metode statis untuk menguji dan mengkonversi karakter dalam berbagai sistem bilangan.

02

## Regular Expressions

Ekspresi reguler untuk validasi input, pencarian pola, dan manipulasi String dengan class Pattern dan Matcher.

03

## Rekursi

Metode yang memanggil dirinya sendiri dengan base case dan recursive step yang jelas.

04

## Fraktal

Aplikasi rekursi dalam dunia visual — bentuk geometris self-similar yang dihasilkan dari pola sederhana yang diulang.

*"Untuk memahami rekursi, Anda harus terlebih dahulu memahami rekursi."*

